

Capítulo 11

Técnicas de Força Bruta

11.1. Objetivos

- Conhecer os mecanismos de geração de wordlist
- Conhecer ferramentas de bruteforce
- Entender o que é boa política de senhas

11.2. Brute Force

Uma das mais conhecidas técnicas de invasão de sistemas é, sem dúvida, o brute force. O método de funcionamento de um ataque desse tipo é muito simples: são geradas várias tentativas de conexão a partir do nome de um provável usuário da máquina alvo. A técnica consiste em gerar várias combinações de senhas para esse usuário, na tentativa de "adivinhar" a senha dele.

Também podemos alternar o nome do usuário, fazendo brute force de usuário e senha. Para isso, podemos obter um arquivo chamado "wordlist", no qual podemos gerar diversas combinações possíveis de senhas para testar com o brute force.

Baseados nisso, veremos algumas ferramentas para realizar esse tipo de teste.

11.3. Wordlist

Uma boa wordlist é fundamental para o sucesso de um ataque de brute force. É possível comprar wordlists, fazer download ou até mesmo gerar listas de palavras que serão usadas pelas ferramentas de brute force.

11.3.1. Download de Wordlist

Wordlists podem ser encontradas em diversos locais, incluindo redes de compartilhamento de arquivos (P2P). Abaixo seguem alguns sites que possuem wordlists para download:

Site:

- <http://www1.harenet.ne.jp/~waring/vocab/wordlists/vocfreq.html>
- <http://www.outpost9.com/files/WordLists.html>
- <http://wordlist.sourceforge.net/>

- <http://rapidshare.com/files/100861231/28GBwordlist.rar>

P2P:

- eMule
- Kazaa
- Torrent

11.4. Geração de Wordlist

Existem diversas ferramentas que auxiliam na geração de uma wordlist. Abaixo listaremos algumas das ferramentas que podem ser usadas para a realização dessa tarefa.



```
# crunch 5 8 12345678 > /tmp/wordlist-numeric
```

Onde:

- 5 - tamanho mínimo da palavra
- 8 - tamanho máximo da palavra
- 12345678 - Caracteres que serão usados para a geração da lista

Temos mais algumas opções a definir com esta ferramenta, mas desta vez vamos tentar criar combinações mais próximas do "mundo real" onde muitas vezes, ou por falta de criatividade ou medo de esquecer a senha as pessoas acabam associando ao próprio nome uma data, casamento, namoro, nascimento, aniversário do papagaio, etc. Vamos ver como poderíamos "adivinhar" a senha o Júnior.



```
# crunch 10 10 1234567890 -t junior@@@@ > /tmp/juniorlist
```

Onde:

- 10 - tamanho mínimo da palavra
- 10 - tamanho máximo da palavra
- 1234567890 - Caracteres que serão usados para a geração da lista

Vamos fazer uso de outra ferramenta para gerar wordlists, no caso, a ferramenta `wyd`. A diferença dessa ferramenta para o `crunch` é que essa utiliza uma maneira mais "inteligente" de gerar as combinações.

Vamos gerar a wordlist com baseado em um arquivo HTML de um site qualquer que fizemos download:



```
# wyd.pl -o /tmp/wordlist ./meu_dump.html
```

Eliminando as palavras repetidas:



```
# cat /tmp/wordlist | sort -u > /tmp/wordlist-inteligente
```

O `wyd` consegue gerar combinações a partir de arquivos em texto puro, html, php, doc, ppt, pdf, odt, ods e odp.

11.5. John The Ripper

O John é um dos utilitários mais conhecidos para decifrar senhas no Linux, pois consegue decifrar algoritmos usados pelo sistema como o MD5 e outras. Toda a configuração do John é feita em um arquivo texto chamado `john.conf` em sistemas

Unix ou john.ini no Windows, por exemplo. Neste arquivo você consegue definir regras para a descoberta de senhas, wordlists, parâmetros para os modos e até definir um novo modo de descoberta de senhas.

Este arquivo é dividido em várias seções. Todas as seções começam com uma linha com seu nome entre colchetes ([]). As opções destas seções são definidas em variáveis de modo bem simples, como em:

- variável = valor

Os nomes de seções e variáveis são case-insensitive, ou seja, SECAO1 e secao1 são a mesma seção e VAR1 e var1 são a mesma variável. Os caracteres # e ; são completamente ignorados, assim como linhas em branco.

Abaixo estão as explicações das opções divididas por seção:

Options:

- Wordlist: A wordlist a ser utilizada pelo JtR. O arquivo pode estar em qualquer lugar, basta especificar o caminho correto nessa variável;
- Idle: Configura o John para usar seu CPU quando este estiver inativo. Diminui o desempenho da quebra da senha, porém não impacta tanto no desempenho de outros programas. O padrão desta opção é N (desabilitado);
- Save: Intervalo no qual o software irá gravar seu progresso para no caso de uma interrupção ele possa recomeçar novamente de onde havia parado;
- Beep: Emite um bip quando uma senha é quebrada.

List.Rules:Single

Nesta seção ficam as regras default do software para a quebra das senhas. São regras como substituição de strings, escrita 1337 e outras.

List.Rules:Wordlist

Nesta seção ficam as regras de substituição de caracteres, modificações de palavras, etc quando se está usando uma wordlist para tentar quebrar as senhas do arquivo.

List.Rules:NT

Nesta seção ficam as regras utilizadas quando se está quebrando senhas cifradas utilizando o algoritmo NTLM (Windows).

Incremental*

Aqui ficam as regras para o tipo de quebra de senhas chamado Incremental (todos os "tipos" de tentativas de quebra de senha que o John utiliza serão explicados mais adiante neste documento).

List.External:*

São alguns filtros pré-definidos para substituição de palavras, eliminação de caracteres indesejados, etc.

11.5.1. MODOS

JtR utiliza alguns modos para que consiga otimizar a quebra da senha. Estes modos são explicados a seguir:

- **Modo Wordlist**

Para utilizar esse método você vai precisar de uma wordlist. Existem vários lugares na Internet que possuem milhares de wordlists disponíveis gratuitamente, é só dar uma olhada no Google que você irá encontrar várias. Para te ajudar, aqui no item "Wordlists" você encontra vários links para wordlists disponíveis na Internet. Lá você também encontra algumas dicas de como organizar a sua lista. Mas vale lembrar que não é bom que você tenha entradas duplicadas na sua lista, o Jhon the Ripper não vai fazer absolutamente nada com a sua wordlist antes de começar a testar as palavras que tem nela.

Este é o modo mais simples suportado pelo John. Para utilizá-lo você só especifica uma wordlist e algumas regras para ele fazer combinações das palavras que ele encontrar na lista que você especificou. Quando utilizando determinados

algoritmos, o JtR se beneficiará se você colocar senhas com tamanhos mais ou menos parecidos perto umas das outras. Por exemplo, seria interessante você colocar as senhas com 8, 6 ou 9 caracteres perto umas das outras na sua wordlist. A wordlist padrão a ser utilizada pelo John é definida no arquivo `john.conf`.

- **Modo Single Crack**

É neste modo que você deveria começar a tentar quebrar uma senha. Aqui, além de várias regras de handling serem aplicadas, o JtR vai utilizar mais informações como o nome completo do usuário e seu diretório home para tentar descobrir qual é a senha. Este modo é muito mais rápido que o modo "Wordlist".

- **Modo Incremental**

Este é o modo mais poderoso do JtR. Nele serão tentadas todas as combinações possíveis de caracteres para tentar quebrar a senha cifrada. Dada a grande quantidade de combinações possíveis, é recomendável que se defina alguns parâmetros (como tamanho da senha ou conjunto de caracteres a serem utilizados) para que você não fique esperando pela senha ser quebrada por muito tempo.

Todos os parâmetros para este modo são definidos no arquivo `john.conf`, nas seções começadas com `Incremental` no nome.

- **Modo External**

Esse modo é bastante complexo. Nele você pode definir regras próprias para o John seguir ao tentar quebrar uma senha. Tais regras são definidas em uma linguagem parecida com a C no arquivo de configuração do programa. Ao ser especificado este modo ao tentar quebrar uma senha na linha de comando, o JtR vai pré-processar as funções que você escreveu para este modo e utilizá-las. A documentação de uso desse modo pode ser obtida em:

<http://www.openwall.com/john/doc/EXTERNAL.shtml>

11.5.2. LINHA DE COMANDO

O John suporta várias opções de linha de comando, geralmente usadas para ativar determinados modos de uso do software. Preste bastante atenção no case das opções, o JtR é case-sensitive! Uma característica muito legal dele é que é possível abreviar as opções da linha de comando desde que não haja ambigüidade (mais ou menos da maneira como ocorre no shell de roteadores Cisco, por exemplo).

Abaixo vou dar uma explicação básica das opções que o John suporta. Se você se esquecer de alguma opção quando estiver utilizando o JtR, é só digitar "john" no terminal e todas as opções serão impressas para você. As opções podem ser definidas utilizando -- ou - e seus parâmetros são definidos utilizando = ou :.

- --single: Define o modo "single" para quebrar as senhas.
- --wordlist=ARQUIVO: Define o modo "wordlist" para quebrar as senhas e define o arquivo ARQUIVO como sendo de onde as senhas serão lidas. Aqui você pode utilizar também a opção --stdin para dizer que as palavras virão da entrada padrão.
- --incremental: Define que será utilizado o modo "incremental" para quebrar as senhas. Opcionalmente você pode definir que tipo de modo incremental será utilizado fazendo --incremental[=MODO].
- --external=MODO: Define que será utilizado o modo external.
- --rules: Habilita as regras para wordlist definidas em john.conf quando se utiliza o modo wordlist.
- --stdout[=LENGTH]: Quando utilizado, faz com que o JtR imprima as possíveis senhas direto na saída padrão ao invés de tentá-las contra um hash. Se você definir o parâmetro LENGTH só serão impressas senhas com caracteres até a quantidade especificada em LENGTH.
- --restore[=NOME]: Faz com que uma sessão que foi interrompida anteriormente continue de onde parou. Se você definir um nome diferente para a sessão, especifique o nome dela na linha de comando

junto com esta opção. A sessão fica gravada na home do John, em um arquivo chamado john.rec.

- `--session=NOME`: Define o nome da sessão que pode ser utilizado com a opção `restore`. A esse nome será automaticamente adicionado a extensão `.rec`.
- `--status[=NOME]`: Mostra o status da última sessão ou, se definido o nome da sessão, da sessão especificada.
- `--make-charset=ARQ`: Gera um arquivo charset para ser utilizado no modo "incremental".
- `--show`: Mostra as senhas do arquivo que você especificou para o JtR que já foram quebradas. Esta opção é especialmente útil quando você tem outra instância do JtR rodando.
- `--test`: Esta opção faz um benchmark de todos os algoritmos compilados no software e os testa para saber se estão funcionando corretamente. Esta opção já foi explicada anteriormente.
- `--users=[-]Nome do usuário ou UID`: Com esta opção você pode especificar para o JtR quais usuário você quer tentar quebrar a senha. Você pode utilizar o nome de usuário ou o UID dele e pode separar vários usuários utilizando uma vírgula. Utilizando o "-" antes do nome do usuário, você faz com que o John ignore aquele usuário ou UID.
- `--groups=[-]GID`: Faz com que o John tente quebrar apenas as senhas dos usuários participantes de um grupo especificado (ou ignorá-los, se você utilizar o "-").
- `--shells=[-]SHELL`: Apenas tenta quebrar as senhas dos usuários cujas shells sejam iguais à que foi especificada por você na linha de comando. Utilizando o "-" você ignora as shells especificadas.
- `--salts=[-]NUMERO`: Deixa você especificar o tamanho das senhas que serão (ou não) testadas. Aumenta um pouco a performance para quebrar algumas senhas, porém o tempo total utilizando esta opção acaba sendo o mesmo.

- `--format=FORMATO`: Permite a você definir o algoritmo a ser usado para quebrar a senha, ignorando a detecção automática do software. Os formatos suportados atualmente são DES, BSDI, MD5, AFS e LM. Você também pode utilizar esta opção quando estiver utilizando o comando `--test`, como já foi explicado anteriormente neste texto.
- `--save-memory=1, 2 ou 3`: Esta opção define alguns níveis para dizer ao John com qual nível de otimização ele irá utilizar a memória. Os níveis variam de 1 a 3, sendo 1 a mínima otimização. Esta opção faz com que o JtR não afete muito os outros programas utilizando muita memória.

11.5.3. USANDO A FERRAMENTA

Para executar o John sobre arquivos de senha de Linux, teremos que passar para ele as senhas estão utilizando o esquema de shadow no sistema. Para isso, utilizaremos o executável "unshadow", que está presente junto com o John:



```
# ./unshadow /etc/passwd /etc/shadow > password
```

Agora podemos executar o John referenciando o nosso arquivo password



```
./john password
Loaded 5 passwords with 5 different salts (FreeBSD MD5 [32/32])
x (x)
x (teste)
abc (teste1)
12345 (sb)
```

Também podemos passar na sintaxe um parâmetro para que o John pegue a lista de palavras de outro arquivo (por exemplo, as wordlists que geramos anteriormente):



```
# ./john -wordfile:/tmp/juniorlist password
```

O John gera dois arquivos de log: o "john.pot" e o "restore". No primeiro arquivo estão as senhas já decifradas, para que em uma nova execução ele não comece tudo do zero. Já o arquivo "restore" irá conter informações sobre o estado de execução do John para continuar executando uma sessão interrompida (por exemplo, quando tecamos "CTRL+C" durante a execução). Se você quiser retomar a execução do ponto onde parou, basta executar:



```
# ./john -restore
```

Também podemos exibir as senhas já descobertas pelo programa usando a opção show:



```
# ./john -show arquivo_passwd
```

É importante ressaltar que o John The Ripper possui módulos adicionais que não são compilados durante uma compilação padrão. Esses módulos podem ser encontrados no próprio site da ferramenta, através do endereço: <http://www.openwall.com/john/>. Os módulos adicionais se encontram no final da página inicial. Como exemplo, podemos citar o módulo para quebrar senhas de Lotus Domino e MySQL.

11.6. THC-Hydra

O hydra é um dos utilitários que abrangem uma grande quantidade de serviços que podem ser alvos de brute force, entre eles: TELNET, FTP, Firebird, HTTP-GET, HTTP-HEAD, HTTPS-GET, HTTP-HEAD, HTTP-PROXY, HTTP-PROXY-NTLM, HTTP-FORM-GET, HTTP-FORM-POST, HTTPS-FORM-GET, HTTPS-FORM-POST, LDAP2, LDAP3, SMB, SMBNT, MS-SQL, MYSQL, POSTGRES, POP3 NTLM, IMAP, IMAP-NTLM, NCP, NNTP, PCNFS, ICQ, SAP/R3, Cisco auth, Cisco enable, SMTP-AUTH, SMTP-AUTH NTLM, SSH2, SNMP, CVS, Cisco AAA, REXEC, SOCKS5, VNC, POP3 e VMware-Auth. Além disso, o hydra fornece suporte a conexões via

proxy. O xhydra é um utilitário gtk para uso do hydra na interface gráfica.

11.6.1. Usando o HydraGTK

Para baixar o HydraGTK, basta acessar o endereço:

<http://freeworld.thc.org/releases/hydra-5.8-src.tar.gz>

Após baixar o arquivo, execute os seguintes comandos para descompactá-lo e compilá-lo:



```
#tar -xvzf hydra-5.8-src.tar.gz
# cd hydra-5.8-src
# ./configure; make; make install
# cd hydra-gtk
# ./configure; make; make install
```

Para executar o programa na interface gráfica, basta chamarmos o binário, que já estará no path do sistema, e informar o path do seu código fonte com o parâmetro -hydra-path:



```
xhydra --hydra-path [caminho-completo]/hydra-5.8-src/
```

11.6.2. Hydra no terminal

Exemplo do hydra sendo usado contra o serviço FTP:



```
# ./hydra -L /tmp/usuarios -P /tmp/pass -o /tmp/resultado -v 192.168.0.100
ftp
[VERBOSE] More tasks defined than login/pass pairs exist. Tasks reduced
to 15.
```

Hydra v4.1 (c) 2004 by van Hauser / THC - use allowed only for legal purposes.

Hydra (http://www.thc.org) starting at 2004-09-28 16:19:21

[DATA] 15 tasks, 1 servers, 15 login tries (l:5/p:3), ~1 tries per task

[DATA] attacking service ftp on port 21

[VERBOSE] Resolving addresses ... done

[STATUS] attack finished for 192.168.0.100 (waiting for childs to finish)

[21][ftp] host: 192.168.0.100 login: x password: 123456

Hydra (http://www.thc.org) finished at 2004-09-28 16:19:29

Em `"/tmp/usuarios"` temos a userlist de ftp, em `"/tmp/pass"` temos a wordlist para os usuários e em `"/tmp/resultado"` o resultado do brute force no serviço.

Vejamos o conteúdo do arquivo de saída do brute force:



```
# cat /tmp/resultado
```

```
# Hydra v4.1 run at 2004-09-28 16:19:21 on 192.168.0.100 ftp
```

```
(hydra -L /tmp/usuarios -P /tmp/pass -t 15 -v -o /tmp/resultado  
192.168.0.100 ftp)
```

```
[21][ftp] host: 192.168.0.100 login: x password: 123456
```

O THC-Hydra também pode ser usado para realizar ataques contra formulários web.

Para isso, podemos usar a opção `http-post-form` ou `http-get-form`, dependendo do método usado para envio dos dados pelo formulário web.

Vamos analisar o comando seguinte:



```
# hydra -l hydra -P password.lst -s 80 <IP> http-post-form  
"/administrador/index.php:usuario=^USER^&senha=^PASS^&submit=Lo  
gin:Incorrect Username"
```

O parâmetro `^USER^` será substituído pelos usuários, no nosso caso, o valor passado para a opção `-l`, no nosso caso o valor `"hydra"`, e o parâmetro `^PASS^` será substituído pelos valores passados na opção `-P`, no nosso caso, uma wordlist chamada `password.lst`.

No exemplo do nosso arquivo que se encontra na vmware de testes, acessível através do endereço `http://<ip_vm>/bf/`, podemos usar o seguinte comando para realizarmos o ataque de bruteforce:



```
root@bt:/tmp# hydra -l admin -P wl.txt -o resultado.txt 192.168.3.106 http-  
get-form  
"/bf/bf.php:usuario=^USER^&senha=^PASS^&submit=Enviar:incorreta"
```

11.7. BruteSSH2

O BruteSSH2 é um script, que pode ser encontrado em C, Perl ou Python, que realiza ataques de força bruta na porta 22 e utiliza wordlists para descobrir senhas.

Para executar é só entrar como root e digitar:



```
# chmod 777 brutessh2.py  
# ./brutessh2.py
```

11.8. Rainbow Crack

RainbowCrack é um programa que gera rainbow tables para serem usadas na quebra de senhas. O RainbowCrack difere dos programas de força bruta convencionais, pois utiliza tabelas previamente criadas, chamadas rainbow tables, para reduzir drasticamente o tempo necessário para quebrar senhas.

Um ótimo programa para utilizarmos para quebrar senhas Windows com rainbow tables é o Ophcrack



```
#aptitude install ophcrack
```

Outro programa, que quebra inclusive hashes MD5, SHA1, SHA2 e etc, é o Cain (que roda em Windows).

- Para baixar Rainbow Tables:

<http://rainbowtables.shmoo.com/>

- Para entender mais:

<http://www.ethicalhacker.net/content/view/94/24/>

11.9. Utilizando o Rainbow Crack para criação de Rainbow Tables

11.9.1. Introdução

RainbowCrack é uma ferramenta cujo objetivo é quebrar hash de senhas.

O método utilizado pela ferramenta é o brute force. Nesse método, todas as senhas em texto plano e seus hashes correspondentes são computados um por um. O hash computado é comparado com o hash alvo. Se um deles for igual, a senha em texto plano é encontrada. Do contrário, o processo continua até finalizar todas as senhas possíveis.

No método time-memory, a tarefa de computar hashes é feita através do armazenamento dos resultados no que chamamos de "rainbow table". Depois disso, os hashes podem ser acessados a partir das rainbow tables sempre que necessário. O processo pré-computacional precisa de muito tempo para criar as chaves que serão posteriormente utilizadas. No entanto, uma vez que esse processo tenha terminado, a performance da rainbow tables pode ser de centenas a milhares de vezes maior do que o método de brute force.

Vamos ver passo a passo como utilizar o software RainbowCrack. Esse software inclui três ferramentas que devem ser usadas em sequência para fazer a coisa funcionar:

- Passo 1: usar o rtgen para gerar as rainbow tables.

- Passo 2: usar o `rtsort` para organizar as rainbow tables geradas pelo `rtgen`.
- Passo 3: usar o `rcrack` para buscar o conteúdo das rainbow tables organizadas pelo `rtsort`.

O processo de buscar o conteúdo no passo final, é equivalente ao processo de quebra de hash. E todas essas ferramentas são utilizadas através da linha de comando.

11.9.2. Passo 1: usar o `rtgen` para gerar as rainbow tables.

O programa `rtgen` precisa de diversos parâmetros para gerar uma rainbow table, e a sintaxe do comando é:



```
rtgen hash_algorithm charset plaintext_len_min plaintext_len_max  
table_index chain_len chain_num part_index
```

Explicação dos parâmetros:

parâmetro	significado
<code>hash_algorithm</code>	O algoritmo dos hashes (lm, ntlm, md5 e assim por diante) usado na rainbow table.
<code>charset</code>	A configuração dos caracteres (charset) do texto plano na rainbow tables. Todos os charsets possíveis estão definidos no arquivo <code>charset.txt</code> .
<code>plaintext_len_min</code> <code>plaintext_len_max</code>	Estes dois parâmetros definem o tamanho possível de todo o texto plano na rainbow tables. Se o charset é numérico, o <code>plaintext_len_min</code> é 1, e o <code>plaintext_len_max</code> é 5, então a string "12345" será incluída na tabela, mas "123456" não.
<code>table_index</code> <code>chain_len</code> <code>chain_num</code> <code>part_index</code>	Estes quatro parâmetros são mais difíceis de explicar em poucas palavras. Ler e compreender o artigo original de Philippe Oechslin (criador do RainbowCrack), pode ajudar a entender o significado exato. O <code>table_index</code> está relacionado ao "reduce function" que é utilizado na rainbow table. O <code>chain_len</code> é o tamanho de cada "rainbow chain" na rainbow table. Uma "rainbow chain"

	configurada como 16 bytes é a menor unidade em uma rainbow table. Uma rainbow tables contém diversas rainbow chains. O chains_num é o número de rainbow chains em uma rainbow table. O parâmetro part_index determina como o "start point" em cada rainbow chain é gerado. Deve ser um número (ou começar com um número).
--	--

Os valores corretos de todos os parâmetros dependem do que você precisa, e selecionar bons parâmetros requer um bom entendimento do algoritmo de time-memory tradeoff.

Uma configuração que funciona está logo abaixo, como um exemplo:

hash_algorithm	lm, ntlm or md5
charset	alpha-numeric = [ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789] ou loweralpha-numeric = [abcdefghijklmnopqrstuvwxyz0123456789]
plaintext_len_min	1
plaintext_len_max	7
chain_len	3800
chain_num	33554432
key space	$36^1 + 36^2 + 36^3 + 36^4 + 36^5 + 36^6 + 36^7 = 80603140212$ Key space é o número de possíveis strings em texto plano para o charset, plaintext_len_min e plaintext_len_max selecionados.
table size	3 GB
success rate	0.999 O algoritmo de time-memory tradeoff é um algoritmos probabilístico. Qualquer que seja os parâmetros selecionados, há sempre probabilidade de que as strings dentro do charset selecionado e o tamanho das strings não seja completamente coberto. A taxa de sucesso é de 99.9% com os parâmetros usados nesse evento.
comandos para gerar as tabelas	Os comandos do rtgen usados para gerar as rainbow tables são: <pre>rtgen md5 loweralpha-numeric 1 7 0 3800 33554432 0 rtgen md5 loweralpha-numeric 1 7 1 3800 33554432 0 rtgen md5 loweralpha-numeric 1 7 2 3800 33554432 0 rtgen md5 loweralpha-numeric 1 7 3 3800 33554432 0 rtgen md5 loweralpha-numeric 1 7 4 3800 33554432 0 rtgen md5 loweralpha-numeric 1 7 5 3800 33554432 0</pre> Se precisar criar uma tabela de hashes ntlm ou lm, substitua o "md5" nos comandos acima por "ntlm" ou "lm". Se precisar de uma tabela com o charset alpha-numeric, substitua o "loweralpha-numeric" nos

comandos acima por "alpha-numeric".

Se uma tabela com hashes lm for criada, tenha certeza de que seu charset seja alpha-numeric ao invés de loweralpha-numeric. O algoritmo lm nunca utiliza caracteres minúsculos como strings.

Agora é hora de criar uma rainbow table.

Altere o diretório corrente em seu terminal de comando para o diretório do RainbowCrack, e execute o comando seguinte:



```
# cd /pentest/passwords/rcrack  
# rtgen md5 loweralpha-numeric 1 7 0 3800 33554432 0
```

Esse comando leva 4 horas para ser completado em um computador com um processador Core2 Duo E7300. É possível parar a execução do mesmo a qualquer momento pressionando Ctrl+C. Da próxima vez que o comando for executado com a mesma linha de comando, ele continuará a partir do ponto em que foi interrompido para continuar com a geração da tabela.

Quando o comando tiver terminado, um arquivo com o nome de "md5_loweralpha-numeric#1-7_0_3800x33554432_0.rt" e tamanho de 512 MB será criado. O nome do mesmo é simplesmente a linha de comando utilizada com os parâmetros interligados, com a extensão "rt". O programa rcrack que será explicado mais a frente, precisa dessas informações para saber quais os parâmetros existentes na rainbow table. Portanto, não renomeie o arquivo.

As demais tabelas podem ser geradas da mesma forma, com os comandos:



```
# rtgen md5 loweralpha-numeric 1 7 1 3800 33554432 0  
# rtgen md5 loweralpha-numeric 1 7 2 3800 33554432 0  
# rtgen md5 loweralpha-numeric 1 7 3 3800 33554432 0  
# rtgen md5 loweralpha-numeric 1 7 4 3800 33554432 0  
# rtgen md5 loweralpha-numeric 1 7 5 3800 33554432 0
```

Finalmente, esses arquivos são gerados:

- md5_loweralpha-numeric#1-7_0_3800x33554432_0.rt 512MB
- md5_loweralpha-numeric#1-7_1_3800x33554432_0.rt 512MB
- md5_loweralpha-numeric#1-7_2_3800x33554432_0.rt 512MB
- md5_loweralpha-numeric#1-7_3_3800x33554432_0.rt 512MB
- md5_loweralpha-numeric#1-7_4_3800x33554432_0.rt 512MB
- md5_loweralpha-numeric#1-7_5_3800x33554432_0.rt 512MB

Agora, o processo de criação da rainbow table está completo.

11.9.3. Passo 2: usar o rtsort para organizar as rainbow tables

As rainbow tables geradas pelo programa rtgen precisam de um pós-processamento para tornar sua consulta mais fácil e rápida. O programa rtsort é utilizado para organizar todas as rainbow chains em uma rainbow table.

Utilize os seguintes comandos:



```
# rtsort md5_loweralpha-numeric#1-7_0_3800x33554432_0.rt  
# rtsort md5_loweralpha-numeric#1-7_1_3800x33554432_0.rt  
# rtsort md5_loweralpha-numeric#1-7_2_3800x33554432_0.rt  
# rtsort md5_loweralpha-numeric#1-7_3_3800x33554432_0.rt  
# rtsort md5_loweralpha-numeric#1-7_4_3800x33554432_0.rt  
# rtsort md5_loweralpha-numeric#1-7_5_3800x33554432_0.rt
```

Cada comando acima, leva cerca de 1 a 2 minutos para completarem sua execução. O programa rtsort gravará a rainbow table organizada por sobre o arquivo original. Não interrompa a execução do comando, do contrário o arquivo original será danificado.

Agora o processo de organização das rainbow tables está completo.

11.9.4. Passo 3: usar o rcrack para buscar o conteúdo das rainbow tables

O programa rcrack é utilizado para acessar as rainbow tables. Ele aceita apenas rainbow tables organizadas.

Assumindo que as rainbow tables organizadas estejam no mesmo diretório do programa, para quebrar hashes únicos a linha de comando será:



```
# rcrack *.rt -h aqui_vai_o_hash_para_ser_quebrado
```

O primeiro parâmetro especifica o caminho para buscar nos arquivos das rainbow tables. Os caracteres "*" e "?" podem ser usados para especificar vários arquivos.

Normalmente isso leva algumas dezenas segundos para finalizar, se a string existir dentro do "range" do charset e tamanho de strings selecionados. Do contrário, leva-se muito mais tempo para buscar por todas as tabelas, apenas para não encontrar nada.

Para quebrar múltiplos hashes, coloque todos os hashes em um arquivo de texto, com um hash por linha. E então especifique o nome do arquivo na linha de comando do programa rcrack:



```
# rcrack *.rt -l arquivo_com_lista_de_hashes
```

Se as rainbow tables que gerou usam o algoritmo lm, o programa rcrack possui um suporte especial para o parâmetro "-f". Um arquivo de dump de hash no formato pwdump é necessário como input para o programa rcrack. O conteúdo do arquivo parecerá com o seguinte:

```
Administrator:500:1c3a2b6d939a1021aad3b435b51404ee:e24106942bf38bcf57a6a4b29016eff6:::
```

```
Guest:501:a296c9e4267e9ba9aad3b435b51404ee:9d978dda95e5185bbda9b3ae00f84b4:::
```

O arquivo pwdump é a saída de utilitários tais como pwdump2, pwdump3 ou outros. E contém os hashes tanto lm quanto ntlm.

Para quebrar hashes lm em arquivos pwddump, use o seguinte comando:



```
# rcrack *.rt -f arquivo_pwddump
```

O algoritmo de hash lm converte todas as letras minúsculas em strings maiúsculas; como resultado disso, todas as strings quebradas através do hashe lm, nunca contém letras minúscula, enquanto que a string original poed conter letras minúsculas. O programa rcrack tentará corrigir isso em hashes ntlm armazenados no mesmo arquivo e exibir a string original.

11.10. Prática dirigida

1. Utilizando o arquivo shadow passado pelo instrutor, vamos executar o John The Ripper para quebrar as senhas.
2. Com o THC-Hydra, vamos executar ataques de força bruta contra um servidor SSH existente na rede alvo.

11.11. OSSTMM Recomenda

- Ataque automatizado de dicionário a pasta de senhas;
- Ataque de força bruta a pasta de senhas;
- Ataque de força bruta em serviços.
- Obter a pasta de senhas do sistema que guarda nomes de usuário e senha;
- Para sistemas Unix, deverão estar em /etc/passwd e/ou /etc/shadow ;
- Para sistemas Unix que realizam autenticações SMB, pode encontrar as

senhas de NT em /etc/smbpasswd;

- Para sistemas NT, deverão estar em /winn/repair/Sam

11.12. Contramedidas

Uma boa política de senhas, de forma a garantir que:

- Senhas são trocadas periodicamente
- A senha deve ter no mínimo 8 caracteres
- A senha deve ser complexa, com caracteres especiais, letras e números
- A mesma senha não pode ser usada dentro de um período mínimo